



COMP – 304

Summer 2026



Creating Responsive and Accessible UIs for Android Apps

Objectives:

- ☐ Discuss the fundamental principles and features of Material Design 3.
- ☐ Identify and utilize various Material Design components to enhance user interfaces.
- ☐ Apply Material Design 3 color schemes to create visually appealing and consistent app themes.
- ☐ Design responsive UIs that adapt to large screens and foldable devices using `WindowSizeClass`.
- ☐ Implement accessibility features to ensure apps are usable by all users, including those with disabilities.



Introduction to Material Design 3

- ❑ Material Design 3 is Google's latest design system.
- ❑ Helps in **creating aesthetically pleasing and functional UIs** for Android apps.
- ❑ Provides **comprehensive guidelines and pre-built components**.
- ❑ Focuses on improving **visual aesthetics and usability**.
- ❑ **Simplifies integration** with Android development environments.



Key Features of Material Design 3

- ❑ Introduction of **dynamic color systems**.
- ❑ **Expanded component library** for enhanced UI construction.
- ❑ **Simplified typography** for clarity and readability.
- ❑ Improved **color schemes** supporting both light and dark themes.
- ❑ Enhanced **customization with Material Theme Builder** tool.



Dynamic Color System

- ❑ **Automatically adjusts app colors** to match user's wallpaper.
- ❑ Ensures **a personalized appearance** for different users.
- ❑ Supported only on Android 12 and above devices.
- ❑ Enhances the user experience by **adapting UI to user preferences**.



Components of Material Design 3

- ❑ Material 3 introduces **new and updated UI components**.
- ❑ Each component designed to **work seamlessly across various devices**.
- ❑ Components include **app bars, buttons, cards**, and more.
- ❑ **Regular updates** and additions to the component library.



Top App Bars in Material 3

- ❑ Key component for navigation and information display.
- ❑ Four types: small, medium, large, and center-aligned.
- ❑ Example code for a center-aligned top app bar:

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun PacktCenterAlignedTopBar() {
    CenterAlignedTopAppBar(
        title = { Text(text = "Packt Publishing") }
    )
}
```



FloatingActionButton (FAB)

- ❑ Represents the primary action in an application.
- ❑ Positioned typically at the bottom right of the screen.
- ❑ Can be customized in size: large, normal, small.
- ❑ Example code for creating a FloatingActionButton:

```
FloatingActionButton(  
    onClick = { /*TODO*/ },  
    content = {  
        Icon(  
            imageVector = Icons.Default.Add,  
            contentDescription = "New Chat"  
        )  
    }  
)
```




Extended FloatingActionButton

- ❑ Variation of the standard FAB, includes additional text or icons.
- ❑ Suitable for more descriptive actions.
- ❑ Example implementation:

```
ExtendedFloatingActionButton(  
    onClick = { /*TODO*/ },  
    content = {  
        Icon(  
            imageVector = Icons.Default.Add,  
            contentDescription = "New Chat"  
        )  
        Text(  
            modifier = Modifier.padding(10.dp),  
            text = "New Chat"  
        )  
    }  
)
```



Bottom App Bars

- ❑ Display navigation items and actions at the bottom of the screen.
- ❑ Ideal for primary destinations within the app.
- ❑ Example code snippet for Bottom App Bar with icons:

```
BottomAppBar(  
    actions = {  
        Icon(imageVector = Icons.Rounded.Home,  
        contentDescription = "Home Screen")  
        Icon(imageVector = Icons.Rounded.ShoppingCart,  
        contentDescription = "Cart Screen")  
        Icon(imageVector = Icons.Rounded.AccountCircle,  
        contentDescription = "Account Screen")  
    }  
)
```



Scaffold Structure in Material Design 3

- ❑ Fundamental layout structure to support top bar, bottom bar, and FAB.
- ❑ Simplifies the layout process ensuring proper alignment and spacing.
- ❑ Example code for implementing a Scaffold with various components:

```
Scaffold(  
    topBar = { PacktSmallTopAppBar() },  
    bottomBar = { PacktBottomNavigationBar() },  
    floatingActionButton = { PacktFloatingActionButton() },  
    content = { paddingValues ->  
        Column(  
            modifier = Modifier  
                .fillMaxSize()  
                .padding(paddingValues)  
                .background(Color.Gray.copy(alpha = 0.1f)),  
            verticalArrangement = Arrangement.Center,  
            horizontalAlignment = Alignment.CenterHorizontally  
        )  
    }  
)
```



Scaffold Structure in Material Design 3

```
{  
    Text(  
        modifier = Modifier.padding(10.dp),  
        text = "Mastering Kotlin for Android Development -  
Chapter 4",  
        textAlign = TextAlign.Center  
    )  
}  
}
```



Adding Material 3 to Your Apps

- ❑ Include the Material 3 library in your project dependencies.
- ❑ Utilize Android Studio templates for quick setup.
- ❑ Example of dependency inclusion for Material Design 3:

```
implementation(libs.androidx.material3)
```



Color Schemes in Material Design 3

- ❑ Extensive **customization options for light and dark themes.**
- ❑ Use the **Material Theme Builder tool** for generating theme colors.
- ❑ Example of setting up color schemes using generated codes:

```
val md_theme_light_primary = Color(0xFF006C49)
```



Customizing Themes with Material Theme Builder

- ❑ Generate and export theme settings for your app.
- ❑ Customize primary, secondary, and tertiary colors.
- ❑ Import generated theme settings into your project.



Implementing Dynamic Color in Apps

- ❑ Set up dynamic colors for adapting UI based on wallpaper.
- ❑ Enable dynamic theming within the app's theme setup.
- ❑ Example of **dynamic color usage**:

```
val dynamicColorScheme = if (darkTheme)
    dynamicDarkColorScheme(context) else
    dynamicLightColorScheme(context)
```




Designing for Large Screens and Foldables

- ❑ Utilize Material 3 guidelines for responsive designs.
- ❑ Implement layouts suitable for varying screen sizes and orientations.
- ❑ Example strategies for different screen types using `WindowSizeClass`.

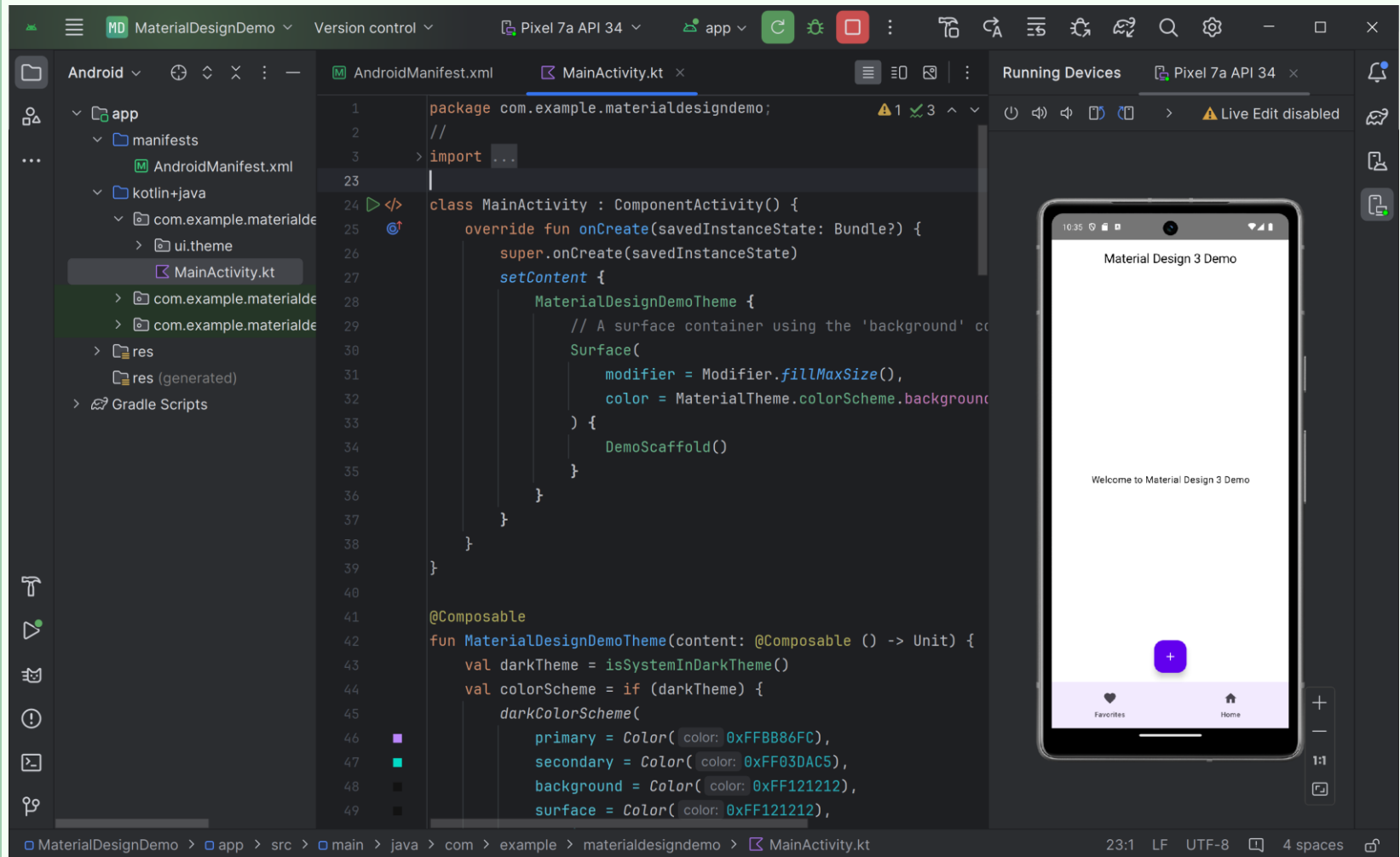


Making Your App Accessible

- ❑ Importance of accessibility in app development.
- ❑ Use of semantics to enhance accessibility.
- ❑ Best practices like adding content descriptions and proper sizing.



MaterialDesignDemo Example





References

- ❑ Textbook Chapter 4
- ❑ <https://developer.android.com/develop/ui/views/theming/look-and-feel>